

Computational Logic of the NMR Pore-Scale Simulation Package

From Bloch–Torrey PDEs to T_2 , T_2 - T_2 Exchange, and D - T_2 Inversion

Yu Zhang
yuz25@univie.ac.at

May 27, 2026, Vienna

Contents

1	Purpose of This Document	2
2	Physical Variables and Units	3
3	Geometry: What Domain Is Simulated?	4
3.1	Two-Triangle Model	4
3.2	Why Large and Small Pores Must Be Kept Separate	4
3.3	Uncoupled Geometry	4
3.4	Coupled Geometry	5
3.5	Throat as an Exchange Path	5
3.6	Segmented Glass-Bead or CT Images	5
4	Water Saturation and Water Allocation	6
4.1	Global Saturation	6
4.2	Allocation Between Small and Large Pores	6
5	Bloch–Torrey Equation	6
5.1	Continuous Equation	6
5.2	Surface Relaxation Boundary	7
5.3	No-Flux Boundary	7
5.4	Dirichlet Boundary	7
6	Why pyGIMLi Is Used	7
6.1	Core Advantage	7
6.2	Mapping the PDE to pyGIMLi	8
7	From Magnetization Field to NMR Signal	8
8	Normalization: Why Some Curves Start from 1 and Others Do Not	9
8.1	Self-Normalized Decay	9
8.2	Total-Pore-Volume Normalization	9
9	One-Dimensional T_2 Inversion	9
9.1	Forward Model	9
9.2	Regularized Non-Negative Least Squares	10
10	Fixed Alpha and L-Curve Regularization	10
10.1	Fixed Alpha	10
10.2	L-Curve	10

11 Coupled and Uncoupled T_2 Calculations	11
11.1 Uncoupled Calculation	11
11.2 Coupled Calculation	11
12 T_2-T_2 Exchange Simulation	11
12.1 Physical Meaning	11
12.2 Signal Generation	12
12.3 Two-Dimensional Inversion	12
13 D-T_2 Simulation	12
13.1 Physical Meaning	12
13.2 PFG Encoding	12
13.3 D - T_2 Inversion	13
14 Control File Logic	13
14.1 Selecting Modules	13
14.2 Selecting Inversion Mode	14
14.3 Selecting Geometry	14
15 Output Files and How to Read Them	14
15.1 T_2 Output	14
15.2 T_2 - T_2 Output	14
15.3 D - T_2 Output	15
16 Common Interpretation Mistakes	15
16.1 Mistake 1: Treating Total Signal as a Pore Component	15
16.2 Mistake 2: Comparing Different Normalizations	15
16.3 Mistake 3: Assuming L-Curve Is Always More Correct	15
16.4 Mistake 4: Ignoring Boundary Markers	15
17 Recommended Code Reading Order	15
18 Summary	16

1 Purpose of This Document

This document explains the lowest-level computational logic of the NMR simulation package. It is written for a new user who needs to understand not only how to run the code, but also why each calculation is performed.

The package simulates pore-scale NMR relaxation in partially saturated porous media. The current main entry point is:

File	Role
Auto_NMR.py	Main program. It runs the PDE simulations, inversion, plotting, and Excel export.
simulation_control.py	User-editable control file. It selects modules, geometry, smoothing, and output folders.
advanced_tools/	Optional scripts for contact-angle batches, CT slices, combined figures, and extra workflow.

The code is designed to compute:

- water distributions at different saturations;

- uncoupled and coupled T_2 decay curves;
- one-dimensional T_2 inversion spectra;
- two-dimensional T_2 - T_2 exchange maps;
- two-dimensional D - T_2 maps;
- fixed regularization and L-curve automatic regularization;
- ideal triangular pores, contact-angle-controlled triangular pores, and segmented glass-bead/CT images.

The most important conceptual chain is:

pore geometry $\rightarrow$ water domain $\rightarrow$ Bloch-Torrey PDE $\rightarrow$ NMR signal $\rightarrow$ inverse Laplace spectrum. (1)

2 Physical Variables and Units

The current code uses micrometers for length and milliseconds for time. This is important because all parameters must be dimensionally consistent.

Quantity	Symbol	Unit in the code
Position	$\mathbf{x}$	μm
Time	t	ms
Diffusion coefficient	D	$\mu\text{m}^2/\text{ms}$
Bulk relaxation time	T_{2B}	ms
Surface relaxivity	ρ	$\mu\text{m}/\text{ms}$
Magnetization	$M(\mathbf{x}, t)$	relative amplitude
NMR signal	$S(t)$	volume-weighted relative amplitude

The default physical parameters in `Auto_NMR.py` are:

```

1 D_scaled, T2B_scaled, rho_scaled = 2.0, 3000.0, 0.005
2 dt = 2.0
3 times = np.arange(0, 1500, dt)
4 t2_axis = np.logspace(0, 4, 150)
```

Their meaning is:

- `D_scaled = 2.0`: water diffusivity in $\mu\text{m}^2/\text{ms}$.
- `T2B_scaled = 3000.0`: bulk relaxation is slow, so wall relaxation dominates.
- `rho_scaled = 0.005`: surface relaxivity at the solid-water boundary.
- `dt = 2.0`: time step for the T_2 PDE.
- `times`: echo-time axis used to record the decay curve.
- `t2_axis`: logarithmic axis used for the inverse T_2 spectrum.

3 Geometry: What Domain Is Simulated?

3.1 Two-Triangle Model

The verified base geometry is a two-pore system: one large triangular pore and one small triangular pore. The current default values are:

```

1 L_large, L_small = 20.0, 8.0
2 depth_large, depth_small = 20.0, 8
3
4 area_large = (np.sqrt(3) / 4) * L_large ** 2
5 area_small = (np.sqrt(3) / 4) * L_small ** 2
6
7 V_large = area_large * depth_large
8 V_small = area_small * depth_small
9 total_pore_volume = V_large + V_small

```

The 2D mesh area is converted to an effective 3D signal volume by multiplying by a depth:

$$V = A_{2D}h. \quad (2)$$

This is why the large and small pores can have different signal weights even when both are represented by 2D meshes.

3.2 Why Large and Small Pores Must Be Kept Separate

The local relaxation rate is strongly related to surface-to-volume ratio:

$$\frac{1}{T_2} \approx \frac{1}{T_{2B}} + \rho \frac{S}{V}. \quad (3)$$

Small pores usually have a larger S/V , so they relax faster and appear at shorter T_2 . Large pores usually have a smaller S/V , so they relax more slowly and appear at longer T_2 .

Therefore, the simulation should export and plot:

- large-pore signal;
- small-pore signal;
- total signal.

The total signal is useful because it resembles what an NMR instrument would measure, but it is not enough for mechanism interpretation.

3.3 Uncoupled Geometry

In the uncoupled case, the large and small pores are solved independently. There is no molecular exchange between them.

$$\Omega_w = \Omega_l \cup \Omega_s, \quad \Omega_l \cap \Omega_s =, \quad (4)$$

with no diffusion path between Ω_l and Ω_s .

The corresponding code is:

```

1 mesh_l = create_pore_with_bubble(...)
2 mesh_s = create_pore_with_bubble(...)
3
4 amp_l_2d = solve_single_decay(mesh_l, ...)
5 amp_s_2d = solve_single_decay(mesh_s, ...)

```

```

6
7 amp_l_vol = amp_l_2d * depth_large
8 amp_s_vol = amp_s_2d * depth_small
9 at_vol = amp_l_vol + amp_s_vol

```

3.4 Coupled Geometry

In the coupled case, the two pores are connected by a throat. The PDE is solved on one connected domain, so magnetization can diffuse from one pore to the other.

```

1 geom_l = create_pore_with_bubble(..., y_shift=L_t / 2, return_geom=True)
2 geom_s = create_pore_with_bubble(..., y_shift=-L_t / 2, return_geom=True)
3 throat = mt.createRectangle(start=[-W_t / 2, -L_t / 2],
4                             end=[W_t / 2, L_t / 2])
5
6 coupled_mesh = mt.createMesh(geom_l + throat + geom_s,
7                               area=0.05,
8                               quality=34)

```

This is a physical coupling model, not a post-processing mixture of two spectra. Coupling occurs because the PDE domain is connected.

3.5 Throat as an Exchange Path

The throat participates in diffusion, but in the current implementation it does not contribute signal volume:

```

1 def build_coupled_cell_volumes(mesh):
2     cell_areas = np.asarray(mesh.cellSizes())
3     centers_y = np.array([c.center().y() for c in mesh.cells()])
4     cell_volumes = np.zeros_like(cell_areas)
5     for i, y in enumerate(centers_y):
6         if y > L_t / 2:
7             cell_volumes[i] = cell_areas[i] * depth_large
8         elif y < -L_t / 2:
9             cell_volumes[i] = cell_areas[i] * depth_small
10        else:
11            cell_volumes[i] = 0.0
12    return cell_volumes

```

This design has a clear purpose:

- the throat allows exchange;
- the throat does not artificially increase the measured pore volume;
- large-pore and small-pore signal components remain interpretable.

3.6 Segmented Glass-Bead or CT Images

For image input, the pore geometry is read from a labeled image. In the current glass-bead example:

Gray value	Phase	NMR role
255	solid bead	no signal, but provides relaxing wall boundary
102	water	PDE domain, produces signal
153	air	no signal
0	outside tube/background	excluded
204	uncertain label	cleaned or reassigned before simulation

The image workflow follows the same logic as the triangular-pore workflow: identify the water domain, identify the solid-water boundary, solve the Bloch–Torrey PDE, and invert the resulting signal.

4 Water Saturation and Water Allocation

4.1 Global Saturation

The global water saturation is:

$$S_w = \frac{V_{\text{water}}}{V_{\text{pore,total}}}. \quad (5)$$

The code converts a target saturation to a target water volume:

```
1 tw_vol = total_pore_volume * sw
```

4.2 Allocation Between Small and Large Pores

The verified base logic fills the small pore first at low saturation. If the requested water volume exceeds the small-pore volume, the remaining water goes into the large pore.

```
1 vw_s, vw_l = (tw_vol, 0.0) if tw_vol <= V_small else (V_small, tw_vol - V_small)
```

Equivalently:

$$V_{w,s} = \begin{cases} V_w, & V_w \leq V_s, \\ V_s, & V_w > V_s, \end{cases} \quad (6)$$

$$V_{w,l} = \begin{cases} 0, & V_w \leq V_s, \\ V_w - V_s, & V_w > V_s. \end{cases} \quad (7)$$

This rule is important because it was part of the previously verified T_2 calculation. Later contact-angle and drainage/imbibition logic should extend this idea, not silently replace it.

5 Bloch–Torrey Equation

5.1 Continuous Equation

The transverse magnetization $M(\mathbf{x}, t)$ in the water-filled pore space satisfies the Bloch–Torrey equation:

$$\frac{\partial M}{\partial t} = D \nabla^2 M - \frac{M}{T_{2B}}. \quad (8)$$

The terms are:

Term	Expression	Meaning
Time evolution	$\partial M / \partial t$	decay of transverse magnetization
Diffusion	$D \nabla^2 M$	molecular motion inside the pore water
Bulk relaxation	$-M / T_{2B}$	relaxation away from the solid wall

The initial condition is:

$$M(\mathbf{x}, 0) = 1, \quad \mathbf{x} \in \Omega_w. \quad (9)$$

5.2 Surface Relaxation Boundary

At the solid-water wall, the code uses a Robin boundary condition:

$$D\nabla M \cdot \mathbf{n} = -\rho M, \quad (10)$$

where $\mathbf{n}$ is the outward normal and ρ is surface relaxivity.

Physically, Eq. 10 means that magnetization is lost at the solid surface. Stronger ρ produces faster relaxation and shorter T_2 .

5.3 No-Flux Boundary

A non-relaxing boundary is represented by:

$$D\nabla M \cdot \mathbf{n} = 0. \quad (11)$$

This means that magnetization does not leave through that boundary. Water-air interfaces are commonly treated this way in the current NMR relaxation calculation.

5.4 Dirichlet Boundary

A Dirichlet condition would prescribe:

$$M = M_0. \quad (12)$$

This is not the main boundary condition for the present relaxation model because solid walls consume magnetization rather than imposing a fixed magnetization value.

6 Why pyGIMLi Is Used

6.1 Core Advantage

The most important numerical difficulty is geometry. Triangular pores, menisci, throats, and segmented CT slices all contain irregular boundaries. A simple pixel-grid finite-difference solver would produce stair-step boundaries and inaccurate surface area. Since T_2 depends strongly on surface-to-volume ratio, this would directly affect the result.

pyGIMLi is useful here because it provides:

- unstructured triangular meshes that conform to irregular pore boundaries;
- convenient geometry construction through `pygimli.meshtools`;
- boundary markers for assigning different physical boundary conditions;
- built-in finite-element PDE solvers;
- direct support for Robin boundary conditions;
- interpolation from mesh nodes to cell centers for signal integration.

6.2 Mapping the PDE to pyGIMLi

The code advances Eq. 8 using an implicit time step. Starting from:

$$\frac{M^{n+1} - M^n}{\Delta t} = D \nabla^2 M^{n+1} - \frac{M^{n+1}}{T_{2B}}, \quad (13)$$

one obtains a linear PDE for the unknown M^{n+1} :

$$M^{n+1} - \Delta t D \nabla^2 M^{n+1} + \frac{\Delta t}{T_{2B}} M^{n+1} = M^n. \quad (14)$$

In the code, one time step is implemented as:

```

1 def _pg_solve_step(mesh, f, D, T_bulk, rho, step_dt):
2     a, b = D * step_dt, -(1.0 + step_dt / T_bulk)
3     bc = {'Robin': {1: rho * step_dt}}
4     f = np.asarray(f, dtype=np.float64)
5     try:
6         return np.asarray(pg.solve(mesh, a=a, b=b, f=f, bc=bc),
7                             dtype=np.float64)
8     except Exception:
9         return np.asarray(pg.solver.solve(mesh, a=a, b=b, f=f, bc=bc),
10                             dtype=np.float64)

```

The variables correspond to:

Code	Meaning	Physical role
<code>a = D * step_dt</code>	diffusion coefficient in the implicit step	controls spatial smoothing
<code>b = -(1 + step_dt/T_bulk)</code>	storage and bulk relaxation term	controls decay per step
<code>f</code>	previous magnetization field M^n	right-hand side
<code>Robin: rho * step_dt</code>	wall relaxation coefficient	surface relaxation

Different PDE libraries use different sign conventions in their weak forms. Therefore, the safest way to reproduce this package is to preserve the coefficient convention used in `_pg_solve_step`.

7 From Magnetization Field to NMR Signal

The measurable signal is the volume integral of magnetization over the water domain:

$$S(t) = \int_{\Omega_w} M(\mathbf{x}, t) dV. \quad (15)$$

In the finite-element mesh, this becomes:

$$S(t) \approx \sum_{c=1}^{N_c} M_c(t) V_c, \quad (16)$$

where $M_c(t)$ is the magnetization at cell center c and V_c is the cell volume.

The implementation is:

```

1 def solve_single_decay(mesh, D, T_bulk, rho, times, dt):
2     u = np.ones(mesh.nodeCount(), dtype=np.float64)
3     amps = []
4     for _ in times:
5         u_cell = np.asarray(pg.interpolate(mesh, u, mesh.cellCenters()))
6         amps.append(np.sum(u_cell * np.asarray(mesh.cellSizes())))
7         u = _pg_solve_step(mesh, u, D, T_bulk, rho, dt)
8     return np.array(amps)

```

Step-by-step:

- `u = ones`: initial magnetization is 1 everywhere in water.
- `pg.interpolate`: node values are interpolated to cell centers.
- `u_cell * cellSizes`: magnetization is weighted by cell area.
- `sum`: the volume integral is approximated by summation.
- `_pg_solve_step`: the PDE advances by one time step.

8 Normalization: Why Some Curves Start from 1 and Others Do Not

Two different normalizations are used for different purposes.

8.1 Self-Normalized Decay

$$\tilde{S}(t) = \frac{S(t)}{S(0)}. \quad (17)$$

This makes every curve start from 1. It is useful for comparing decay rates, but it hides the difference in water amount.

8.2 Total-Pore-Volume Normalization

$$\hat{S}(t) = \frac{S(t)}{V_{\text{pore,total}}}. \quad (18)$$

This makes low-saturation curves start at lower amplitudes. It is useful for comparing signal quantity across saturations.

The code uses:

```
1 axs[1, i].plot(times, amp_l_vol / total_pore_volume, label='Large')
2 axs[1, i].plot(times, amp_s_vol / total_pore_volume, label='Small')
3 axs[1, i].plot(times, at_vol / total_pore_volume, label='Total')
```

When comparing figures, the normalization must be checked first. Otherwise, two correct plots may appear inconsistent.

9 One-Dimensional T_2 Inversion

9.1 Forward Model

The measured decay is approximated as a sum of exponentials:

$$S(t_j) \approx \sum_{i=1}^N p_i \exp\left(-\frac{t_j}{T_{2,i}}\right). \quad (19)$$

The matrix form is:

$$\mathbf{s} \approx \mathbf{A}\mathbf{p}, \quad (20)$$

where:

$$A_{ji} = \exp\left(-\frac{t_j}{T_{2,i}}\right). \quad (21)$$

The unknown vector $\mathbf{p}$ is the T_2 spectrum. Each p_i must be non-negative.

9.2 Regularized Non-Negative Least Squares

Inverse Laplace transformation is ill-posed. Many different spectra can fit almost the same decay curve. To stabilize the result, the code solves:

$$\min_{p \geq 0} \|Ap - s\|_2^2 + \alpha^2 \|Lp\|_2^2, \quad (22)$$

where L is a second-derivative smoothing matrix.

The code is:

```

1 def invert_t2_fixed(times, signal, t2_bins, alpha):
2     if np.max(signal) <= 1e-12:
3         return np.zeros(len(t2_bins))
4     A = np.exp(-np.outer(times, 1.0 / t2_bins))
5     n = len(t2_bins)
6     L = make_second_derivative_matrix(n)
7     spectrum, _ = nnls(
8         np.vstack((A, L * alpha)),
9         np.concatenate((signal / signal[0], np.zeros(n)))
10    )
11    return spectrum

```

The augmented system:

$$\begin{bmatrix} A \\ \alpha L \end{bmatrix} p \approx \begin{bmatrix} s \\ 0 \end{bmatrix} \quad (23)$$

means that the result must both fit the signal and remain smooth.

10 Fixed Alpha and L-Curve Regularization

10.1 Fixed Alpha

Fixed alpha means the user manually chooses α :

```

1 "inversion": {
2     "mode": "fixed",
3     "alpha": 1.0,
4     "image_mode": "l_curve",
5 }

```

This is recommended for clean synthetic simulations because every case uses the same smoothing strength. That makes comparisons more controlled.

10.2 L-Curve

The L-curve method chooses α automatically. For each trial α , it computes:

$$r(\alpha) = \|Ap_\alpha - s\|_2, \quad (24)$$

$$\eta(\alpha) = \|Lp_\alpha\|_2. \quad (25)$$

The chosen α is usually near the corner of the curve in the $\log r$ - $\log \eta$ plane. This corner balances data fitting and smoothness.

L-curve regularization is especially useful for segmented image or glass-bead data because these data contain boundary roughness, label noise, and discretization artifacts.

11 Coupled and Uncoupled T_2 Calculations

11.1 Uncoupled Calculation

The uncoupled model computes the large-pore and small-pore decays separately:

$$S_{\text{unc}}(t) = S_l(t) + S_s(t). \quad (26)$$

The corresponding spectra are also computed separately:

$$P_{\text{unc}}(T_2) = P_l(T_2) + P_s(T_2). \quad (27)$$

This is why the package exports Large, Small, and Total columns.

11.2 Coupled Calculation

The coupled model solves one connected PDE:

$$\Omega_w = \Omega_l \cup \Omega_t \cup \Omega_s, \quad (28)$$

where Ω_t is the throat.

The code records total, large-region, and small-region amplitudes:

```

1 def solve_coupled_decay_physics(mesh, D, T_bulk, rho, times, dt):
2     u = np.ones(mesh.nodeCount(), dtype=np.float64)
3     amps_tot, amps_l, amps_s = [], [], []
4
5     centers_y = np.array([c.center().y() for c in mesh.cells()])
6     idx_l = centers_y >= 0
7     idx_s = centers_y < 0
8     cell_volumes = build_coupled_cell_volumes(mesh)
9
10    for _ in times:
11        u_cell = np.asarray(pg.interpolate(mesh, u, mesh.cellCenters()))
12        amps_tot.append(np.sum(u_cell * cell_volumes))
13        amps_l.append(np.sum(u_cell[idx_l] * cell_volumes[idx_l]))
14        amps_s.append(np.sum(u_cell[idx_s] * cell_volumes[idx_s]))
15        u = _pg_solve_step(mesh, u, D, T_bulk, rho, dt)
16
17    return np.array(amps_tot), np.array(amps_l), np.array(amps_s)

```

The strength of coupling depends on throat width, throat length, diffusion coefficient, relaxation time scale, and pore-size contrast.

12 T_2 - T_2 Exchange Simulation

12.1 Physical Meaning

A T_2 - T_2 exchange experiment contains three stages:

1. first relaxation period t_1 ;
2. mixing time t_m ;
3. second relaxation period t_2 .

If water molecules stay in the same pore environment, the map mainly contains diagonal intensity:

$$T_{2,1} = T_{2,2}. \quad (29)$$

If water molecules move between different pore environments, off-diagonal intensity appears:

$$T_{2,1} \neq T_{2,2}. \quad (30)$$

12.2 Signal Generation

The code creates a two-dimensional signal matrix:

$$S(t_1, t_2). \quad (31)$$

```

1 def solve_t2_t2_exchange(mesh, D, T2B, rho, times1, tm, times2):
2     S = np.zeros((len(times1), len(times2)))
3     for i, t1 in enumerate(times1):
4         u = np.ones(mesh.nodeCount(), dtype=np.float64)
5
6         # evolve during t1
7         # evolve during tm
8         # record during t2
9
10        for j, t2 in enumerate(times2):
11            u_cell = np.asarray(pg.interpolate(mesh, u, mesh.cellCenters()))
12            S[i, j] = np.sum(u_cell * np.asarray(mesh.cellSizes()))
13        ...
14    return S

```

12.3 Two-Dimensional Inversion

The inversion model is:

$$S(t_{1,j}, t_{2,k}) \approx \sum_m \sum_n F_{mn} \exp\left(-\frac{t_{1,j}}{T_{2,m}}\right) \exp\left(-\frac{t_{2,k}}{T_{2,n}}\right). \quad (32)$$

The code builds the two-dimensional kernel using Kronecker products:

```

1 A1 = np.exp(-np.outer(times1, 1.0 / t2_bins))
2 A2 = np.exp(-np.outer(times2, 1.0 / t2_bins))
3 K = np.kron(A2, A1)

```

The two-dimensional spectrum is solved by NNLS:

```

1 F_vec, _ = nnls(A_aug, b_aug)
2 return F_vec.reshape((n, n), order='F')

```

13 D - T_2 Simulation

13.1 Physical Meaning

A D - T_2 map shows how apparent diffusion coefficient and relaxation time are correlated. It answers:

For water with a certain T_2 , what apparent diffusion coefficient does it have?

Large pores tend to have less restricted diffusion and longer T_2 . Small pores tend to have stronger restriction and shorter T_2 .

13.2 PFG Encoding

The code uses a simplified pulsed-field-gradient encoding:

1. apply a first phase factor $\exp(iqx)$;

2. evolve the magnetization field during diffusion time Δ ;
3. apply the opposite phase factor $\exp(-iqx)$;
4. continue relaxation until each echo time TE ;
5. record $S(b, TE)$.

In the code:

```

1 q = math.sqrt(max(float(bval), 0.0) / max(delta_diff, 1e-12))
2 phase_plus = np.exp(1j * q * coord)
3 phase_minus = np.conj(phase_plus)
4
5 u = phase_plus.copy()
6 u = evolve_complex_field(mesh, u, delta_diff, D, T_bulk, rho, dt_pde)
7 u *= phase_minus

```

The approximate relation is:

$$b = q^2 \Delta. \quad (33)$$

13.3 D - T_2 Inversion

The forward model is:

$$S(b, TE) \approx \int \int F(D_{\text{app}}, T_2) \exp(-bD_{\text{app}}) \exp\left(-\frac{TE}{T_2}\right) dD_{\text{app}} dT_2. \quad (34)$$

The discrete matrix form is:

$$\mathbf{S} \approx \mathbf{A}_D \mathbf{F} \mathbf{A}_T^T. \quad (35)$$

The code builds:

```

1 AD = np.exp(-np.outer(b_axis, d_bins))
2 AT = np.exp(-np.outer(te_axis, 1.0 / t2_bins))
3 K = np.kron(AT, AD)

```

Then it solves a regularized NNLS problem in two dimensions.

14 Control File Logic

The user usually changes `simulation_control.py`, not the main program.

14.1 Selecting Modules

```

1 "modules": {
2     "uncoupled_t2": True,
3     "coupled_t2": True,
4     "t2_t2": True,
5     "dt2": True,
6 }

```

Switch	Meaning
uncoupled_t2	independent large/small pore T_2 calculation
coupled_t2	connected-pore T_2 calculation
t2_t2	T_2 - T_2 exchange map
dt2	D - T_2 map

14.2 Selecting Inversion Mode

```

1 "inversion": {
2   "mode": "fixed",
3   "alpha": 1.0,
4   "image_mode": "l_curve",
5 }

```

Recommended usage:

- use `fixed` for ideal triangular synthetic cases;
- use `l_curve` for segmented image or glass-bead cases;
- keep the same `alpha` when comparing a controlled batch of ideal simulations.

14.3 Selecting Geometry

```

1 "geometry": {
2   "mode": "verified_triangle",
3   "triangle_angles_deg": [60.0, 60.0, 60.0],
4   "contact_angle_deg": 0.0,
5   "process": "Drainage",
6   "slice_path": None,
7 }

```

The stable main-program geometry is `verified_triangle`. Contact-angle batch calculations and CT-slice workflows are available in `advanced_tools/`.

15 Output Files and How to Read Them

15.1 T_2 Output

Output	Meaning
Triangle_Raw_Decay.xlsx	raw decay curves for large, small, and total signals
T2_Inversion_Detailed_2.5D.xlsx	one-dimensional T_2 spectra
NMR_T2_Figures/	figures generated by the script

When reading a T_2 figure, check:

- peak position, which indicates relaxation time;
- peak area, which indicates signal amount;
- whether the plotted curve is large pore, small pore, or total;
- whether the vertical axis is self-normalized or total-volume-normalized.

15.2 T_2 - T_2 Output

Output	Meaning
T2_T2_Exchange_Map.xlsx	two-dimensional exchange spectrum

Diagonal peaks indicate little apparent exchange or exchange between similar environments. Off-diagonal peaks indicate exchange between different relaxation environments.

15.3 D - T_2 Output

Output	Meaning
DT2_Uncoupled_Map.xlsx	uncoupled D - T_2 spectrum
DT2_Coupled_Map.xlsx	coupled D - T_2 spectrum
DT2_Uncoupled_Signal_b_TE.xlsx	uncoupled raw $S(b, TE)$ signal
DT2_Coupled_Signal_b_TE.xlsx	coupled raw $S(b, TE)$ signal

The T_2 direction describes relaxation environment. The D_{app} direction describes diffusion restriction.

16 Common Interpretation Mistakes

16.1 Mistake 1: Treating Total Signal as a Pore Component

The total signal is:

$$S_{total} = S_l + S_s. \quad (36)$$

It is not the large pore or the small pore. Mechanistic interpretation requires separate large/small components.

16.2 Mistake 2: Comparing Different Normalizations

A self-normalized curve starts from 1:

$$S(t)/S(0). \quad (37)$$

A volume-normalized curve starts from the water-volume fraction:

$$S(t)/V_{pore,total}. \quad (38)$$

These two plots answer different questions.

16.3 Mistake 3: Assuming L-Curve Is Always More Correct

L-curve regularization is a robust automatic choice for noisy data, but it is not a physical law. For clean synthetic comparisons, fixed alpha is often better because it keeps the inversion protocol identical across cases.

16.4 Mistake 4: Ignoring Boundary Markers

Surface relaxation must be applied only to the correct solid-water walls. If boundary markers are wrong, the PDE will relax magnetization at incorrect locations, and the resulting T_2 spectrum will be physically misleading.

17 Recommended Code Reading Order

A new user should read the code in this order:

1. `simulation_control.py`: what will be simulated?
2. top of `Auto_NMR.py`: units and global parameters.
3. `create_pore_with_bubble`: geometry and water/air morphology.
4. `_pg_solve_step`: one implicit PDE step.

5. `solve_single_decay`: uncoupled single-pore decay.
6. `solve_coupled_decay_physics`: coupled two-pore decay.
7. `invert_t2_fixed`: one-dimensional T_2 inversion.
8. `smart_invert_lcurve`: fixed-alpha versus L-curve selection.
9. `solve_t2_t2_exchange`: T_2 - T_2 signal generation.
10. `invert_t2_t2_nnls`: T_2 - T_2 inversion.
11. `solve_dt2_signal_pfg`: PFG-based D - T_2 signal generation.
12. `invert_d_t2_nnls`: D - T_2 inversion.
13. `export` section: Excel and figure outputs.

18 Summary

The package performs pore-scale NMR simulation by combining three layers:

1. **Geometry and saturation**: define where water exists.
2. **pyGIMLi finite-element PDE**: solve diffusion and surface relaxation in the actual pore geometry.
3. **NNLS inversion**: convert simulated signals into T_2 , T_2 - T_2 , and D - T_2 spectra.

The central physical equation is the Bloch–Torrey equation with Robin surface relaxation. The central numerical tool is pyGIMLi, because it can solve PDEs on irregular pore geometries with explicit boundary markers. The central inversion method is regularized non-negative least squares, with either manually selected fixed alpha or automatically selected L-curve alpha.

In short:

The simulation first asks where the water is, then computes how magnetization decays inside that water domain, and finally inverts the decay into NMR spectra.